



Beverage Warehouse Inventory Management System

Multi-Warehouse Beverage Distribution, Barcode Scanning, Expiry Tracking & Cold Chain Management

Stack: Go + Fiber v3 → PostgreSQL || Vue 3 + TypeScript + PrimeVue

Scale: Mid-size beverage distributor, multi-warehouse, role-based access

Duration: 8–12 weeks (single dev) / 4–6 weeks (2-person team)

Table of Contents

1. Project Overview
2. Background / Introduction
3. Goals and Objectives
4. Our Service
5. Project Scope
6. Software Requirements
7. Technology Stack
8. System Architecture / Design
9. Work Plan / Timeline
10. Budget and Resources
11. Risk Management
12. Expected Output / Deliverables
13. Conclusion
14. Meet Our Team

1. Project Overview

The Beverage Warehouse Inventory Management System (BWIMS) is a full-stack web application designed specifically for beverage distributors and wholesale operations. It manages inventory across multiple warehouses, tracks stock movements, enforces expiry-based picking (FEFO), and supports barcode scanning — all tailored to the unique demands of the beverage industry.

Beverage distribution involves thousands of SKUs across brands, flavors, sizes, and packaging types. Each product carries batch numbers and expiration dates that must be strictly tracked to prevent spoilage and ensure regulatory compliance. The system provides real-time inventory snapshots per warehouse, zone, and location — with automated alerts for near-expiry stock and low-stock thresholds.

The application supports four stock movement types (receive, pick, transfer, adjust) within database transactions, maintains an immutable audit trail, and uses FIFO/FEFO cost layers for accurate valuation. Barcode scanning via phone camera or USB scanner enables warehouse floor staff to complete receive and pick operations in under 10 seconds. The system is built for mid-size beverage distributors with: warehouse managers overseeing operations, pickers handling floor-level picking, admin users managing configuration and users, and viewers needing read-only access for auditing and planning.

2. Background / Introduction

Problem Statement

Beverage distributors and wholesalers manage enormous inventories spanning soft drinks, water, juices, energy drinks, alcoholic beverages, and more — each with multiple brands, flavors, sizes, and packaging types (cans, bottles, kegs, cartons). Many still rely on spreadsheets, paper logs, or generic inventory software that cannot handle beverage-specific requirements: batch/lot expiry tracking, temperature zone management, and FEFO (First Expired, First Out) picking logic. This leads to product spoilage, regulatory non-compliance, stock discrepancies, slow order fulfillment, and revenue loss from expired inventory write-offs.

Need for the Software

There is a clear need for a beverage-specific warehouse management system that provides: real-time visibility of stock across multiple distribution centers, mandatory lot/batch tracking with automated expiry alerts, FEFO-based pick logic to minimize waste and spoilage, temperature zone mapping within warehouses, barcode scanning for fast floor operations, and complete audit trail for regulatory compliance and reconciliation.

Target Users

- **Warehouse Managers:** Oversee beverage stock levels across distribution centers, approve adjustments, monitor expiry alerts, generate inventory valuation and wastage reports
- **Pickers / Floor Staff:** Receive incoming beverage deliveries, pick orders using FEFO logic via barcode scanners, transfer stock between temperature zones
- **Administrators:** System configuration, user management, beverage product catalog setup (brands, flavors, sizes, categories), warehouse/zone/location setup
- **Viewers / Auditors:** Read-only access to inventory data for compliance audits, financial planning, and supplier reconciliation

3. Goals and Objectives

Goal: To develop a beverage-specific, multi-warehouse inventory management system that provides real-time stock tracking, FEFO-based picking, expiry management, and barcode-driven operations for beverage distributors.

Objectives:

- Build a secure JWT-based authentication system with 4 distinct roles (admin, manager, picker, viewer)
- Implement full CRUD for warehouses with temperature zone mapping (ambient, chilled, frozen), locations, beverage products (brand, flavor, size, packaging), and categories
- Support mandatory lot/batch tracking for all beverage products, including expiration date, batch number, and production date
- Implement FEFO (First Expired, First Out) pick logic to minimize product spoilage and waste
- Support four stock movement types: receive, pick, transfer, and adjust — all within database transactions
- Automated expiry alerts — notify managers when products are approaching expiration (30/14/7 days)
- Integrate barcode scanning (camera + USB scanner) for sub-10-second receive/pick operations
- Implement FIFO cost layer consumption for accurate beverage inventory valuation
- Maintain an immutable, append-only audit trail in stock_movements for regulatory compliance
- Support 50,000+ beverage SKU rows with virtual scrolling in the UI
- Deliver API response times under 200ms for 95th percentile of requests

4. Our Service

The Beverage Warehouse Inventory Management System provides the following core services tailored to beverage distribution:

- **Beverage Product Catalog:** Full product catalog organized by brand, flavor, size (ml/L), packaging type (can, bottle, keg, carton), and hierarchical categories (soft drinks, water, juice, energy drinks, alcoholic beverages, mixers)
- **Multi-Warehouse with Temperature Zones:** Create and manage multiple distribution centers, each with temperature-mapped zones (ambient, chilled, frozen) and hierarchical locations (zone → aisle → rack → bin) for precise stock placement
- **Lot/Batch & Expiry Tracking:** Every inventory receipt is linked to a lot/batch with production date, expiration date, and batch number — mandatory for all beverage products to ensure traceability
- **FEFO Picking Logic:** When picking stock, the system automatically selects the batch with the nearest expiration date (First Expired, First Out) to minimize spoilage and waste
- **Real-Time Inventory Snapshots:** View stock levels by warehouse, zone, product, and lot — with expiry status indicators (fresh, approaching expiry, expired)
- **Expiry Alert System:** Automated notifications at 30, 14, and 7 days before batch expiration — enabling proactive discounting, redistribution, or disposal decisions
- **Stock Movement Operations:** Four movement types (receive, pick, transfer, adjust) all executed within database transactions — picks automatically consume lots in FEFO order
- **Barcode Scanning:** Phone camera scanning via @teckel/vue-barcode-reader plus USB scanner (keyboard wedge) support — scan a beverage barcode and complete operations on the same screen
- **FIFO Cost Tracking:** Automatic cost layer consumption during picks (oldest received first) for accurate inventory valuation — critical for financial reporting in beverage distribution
- **Reporting & Dashboard:** Inventory valuation reports, expiry reports (by date range), movement summaries, product velocity analytics, wastage tracking, CSV exports, and

KPI dashboards

- **Role-Based Access Control:** Four distinct roles ensuring viewers can't modify, pickers can only perform movements, managers oversee operations, and admins have full system control
- **Immutable Audit Trail:** Every stock movement is append-only — never updated or deleted — enabling complete regulatory compliance and reconciliation

5. Project Scope

Inclusions (What the system will provide)

- **Authentication:** JWT (RS256) auth with login, registration, token refresh, and role-based middleware
- **User Management:** Admin-only CRUD for user accounts with role assignment (admin, warehouse_manager, picker, viewer)
- **Warehouse & Zone Management:** CRUD for warehouses and temperature-mapped zones (ambient, chilled, frozen) with hierarchical locations
- **Beverage Product Catalog:** Full product CRUD with SKU, barcode, brand, flavor, size (ml/L), packaging type, and category — organized by beverage type
- **Lot/Batch Management:** Mandatory lot tracking for all beverage products — batch number, production date, expiration date
- **Inventory Management:** Stock snapshots by warehouse/zone/product/lot, low-stock alerts, expiry alerts (30/14/7 days), per-warehouse summaries
- **Stock Movements:** Receive, pick (FEFO), transfer, adjust — all with transactional updates and FIFO cost layer tracking
- **Barcode Scanning:** Camera-based (phone) and USB scanner (keyboard wedge) integration for GS1/EAN-13 barcodes standard on beverage products
- **Barcode Generation:** Backend barcode label generation using Go boombuler library for shelf/bin labels
- **Expiry Management:** Automated expiry date tracking, near-expiry alerts, expired stock quarantine workflow
- **Reporting:** Inventory valuation (FIFO), expiry reports, movement summaries, product velocity, wastage reports, CSV export
- **Dashboard:** KPIs, expiry alerts widget, low-stock alerts, recent movements, inventory by temperature zone
- **Security:** Rate limiting, request logging, comprehensive input validation, error standardization

- **Testing:** Go integration tests (database layer), Vue component tests (Vitest), E2E tests (Playwright)

Exclusions (Not included in this version)

- Temperature sensor/hardware integration (IoT thermometers) — manual zone assignment only
- Offline PWA support with IndexedDB sync
- ERP integrations (SAP, Oracle NetSuite)
- Real-time WebSocket/SSE live updates across users
- Multi-tenant SaaS architecture — single beverage distributor deployment
- Advanced accounting integration (QuickBooks, Xero)
- Physical barcode printer hardware integration
- Route optimization for delivery trucks
- E-commerce or direct-to-consumer order integration

6. Software Requirements

a. Functional Requirements

#	Feature	Description
FR-1	User Registration & Login	JWT (RS256) authentication with secure login, registration, and token refresh
FR-2	Role-Based Access	4 roles: admin, warehouse_manager, picker, viewer — enforced by middleware
FR-3	User Management	Admin CRUD for user accounts with role assignment and activation/deactivation
FR-4	Warehouse CRUD	Create, read, update, delete warehouses with name, code, address, and active status
FR-5	Location Management	Hierarchical locations (zone/aisle/rack/bin) within each warehouse, with barcode tags
FR-6	Product CRUD	Manage products with SKU, barcode, name, category, unit, and lot tracking flag
FR-7	Category Management	Hierarchical product categories (parent/child)
FR-8	Barcode Lookup	Look up product by barcode via <code>GET /api/products/by-barcode/:code</code>
FR-9	Product Search	Search products by SKU, name, or barcode with category filtering
FR-10	Inventory Snapshot	Query stock by warehouse, product, and location with aggregated quantities

FR-11	Low-Stock Alerts	Query products below threshold quantity per warehouse with configurable thresholds
FR-12	Expiry Alerts	Automated alerts at 30, 14, and 7 days before batch expiration — with quarantine flag for expired stock
FR-13	Receive Inventory	Receive beverage stock into a location — mandatory lot/batch (batch#, production date, expiry date), updates stock, inserts cost layer
FR-14	Pick Inventory (FEFO)	Pick stock from a location — automatically selects batch with nearest expiry date (FEFO), consumes FIFO cost layers, decrements stock
FR-15	Transfer Stock	Move stock between warehouses/zones/locations — validates source has sufficient quantity, preserves lot/batch data
FR-16	Stock Adjustment	Correct stock counts with reason (damage, spillage, expiry write-off) — creates adjustment movement
FR-17	Movement History	Query all stock movements with filters (type, product, date range, location, lot/batch)
FR-18	FIFO Cost Tracking	Automatic cost layer creation on receive and consumption on pick (oldest layers first)
FR-19	FEFO Pick Logic	Picks automatically consume lots by nearest expiration date; prevents picking expired lots
FR-20	Barcode Generation	Server-side barcode image generation for product and shelf/bin labels
FR-21	Camera Barcode Scan	Phone camera scanning for GS1/EAN-13 barcodes via @teckel/vue-barcode-reader
FR-22	USB Scanner Input	Keyboard wedge scanner support via composable (useScanner.ts)
FR-23	Inventory Valuation	FIFO-based valuation report per warehouse and zone
FR-24	Expiry Report	Report of stock by expiry status (fresh, approaching expiry, expired) with batch details

FR-25	Movement Summary	Aggregated movement report by type and date range
FR-26	Product Velocity	Top-moving beverage products analysis by brand, category, and SKU
FR-27	CSV Export	Export reports to CSV format
FR-28	Dashboard	KPIs, expiry alerts widget, low-stock alerts, recent movements, stock by temperature zone

b. Non-Functional Requirements

#	Category	Requirement
NFR-1	Performance	API responses under 200ms for 95th percentile
NFR-2	Scalability	PrimeVue DataTable must handle 50,000+ product rows with virtual scroll
NFR-3	Security	JWT RS256 tokens, bcrypt password hashing, rate limiting, HTTPS enforcement
NFR-4	Data Integrity	All stock movements are append-only (immutable); stock updates are transactional
NFR-5	Usability	Barcode scan-to-operate in under 10 seconds; mobile-responsive for floor staff
NFR-6	Reliability	Database-level UNIQUE constraints prevent duplicate stock rows; FIFO is deterministic
NFR-7	Accessibility	Role boundaries: viewer (read-only), picker (movements only), manager (CRUD + reports), admin (full)
NFR-8	Testability	Integration tests for database layer; component tests (Vitest); E2E tests (Playwright)

7. Technology Stack

Category	Technology	Rationale
Backend Language	Go 1.22+	High performance, excellent concurrency, single binary deployment
Web Framework	Fiber v3	Express-inspired, fast HTTP framework for Go
SQL Codegen	sqlc + pgx v5	Compile-time type safety, full SQL power (CTEs, window functions), zero ORM magic
Database	PostgreSQL 16	ACID compliance, rich SQL features, reliable for inventory systems
Migrations	golang-migrate	Schema versioning with up/down migrations
Auth	JWT RS256 + bcrypt	Stateless auth, no Redis dependency, works with mobile scanners
Barcode Gen	boombuler/barcode (Go)	Server-side barcode image generation
Frontend	Vue 3 + TypeScript + Vite	Modern reactive framework, type-safe, fast build tooling
UI Library	PrimeVue 4	Best DataTable for 10k+ rows (virtual scroll, export, filtering)
State Management	Pinia	TypeScript-native, Composition API style, modular stores
Form Validation	VeeValidate + Zod	Schema-based validation with TypeScript inference

HTTP Client	Axios	Interceptors for JWT refresh, error handling
Barcode Reader	@teckel/vue-barcode-reader	Camera-based barcode scanning in the browser
Charts	ApexCharts	Interactive charts for dashboard and reports
Testing	Go testing + Vitest + Playwright	Unit, integration, component, and E2E testing
Containerization	Docker + Docker Compose	Consistent dev environment with PostgreSQL + API
Deployment	Go API + Vue SPA via nginx/CDN	Separate deployments for independent scaling

8. System Architecture / Design

Architecture: Clean Layered (Backend) + SPA (Frontend)

Backend Structure

```

cmd/server/main.go          # Entry point, DI wiring
internal/
  config/config.go          # Environment variables, config struct
  handler/                  # HTTP handlers (thin layer)
    auth.go, user.go, warehouse.go, product.go,
    inventory.go, movement.go, report.go, barcode.go
  service/                  # Business logic
    auth.go, user.go, warehouse.go, product.go,
    inventory.go, movement.go, cost.go, report.go
  repository/               # sqlc-generated + custom queries
    gen/                    # sqlc output (do not edit)
    custom/                 # Hand-written complex queries
  middleware/
    auth.go, rbac.go, logging.go, ratelimit.go
  router/router.go          # Route registration
pkg/
  entity/                   # Shared domain structs
  api/                      # Request/Response DTOs
  lib/                      # Helpers (barcode, pagination, etc.)
  migrations/               # *.up.sql, *.down.sql

```

Layers: Handler → Service → Repository → PostgreSQL

DI: Constructor injection (no framework)

Error Model: `AppError{Code, Message, Err}` with global Fiber ErrorHandler

Frontend Structure

```

frontend/src/
  api/          client.ts, auth.ts, products.ts, inventory.ts, movements.ts
  components/   LocationTree.vue, BarcodeScanner.vue, StockMovementForm.vue, L
  composables/  useAuth.ts, useScanner.ts, useCharts.ts, usePagination.ts, us

```

layouts/
router/
stores/
views/

DefaultLayout.vue, AuthLayout.vue
index.ts, guards.ts
auth.ts, products.ts, inventory.ts, warehouses.ts, movements.ts
Login.vue, Dashboard.vue, Warehouses/, Products/, Inventory/,

Database Schema (Key Tables)

Table	Key Columns	Notes
warehouses	id, name, code, address, is_active	Multi-warehouse distribution centers
locations	id, warehouse_id, temp_zone (ambient/chilled/frozen), aisle, rack, shelf, barcode, is_pickable	Temperature-mapped storage zones
categories	id, name, parent_id	Beverage categories (soft drinks, water, juice, energy, alcohol, etc.)
products	id, sku, barcode (UNIQUE), name, brand, flavor, size_ml, packaging, category_id, is_lot_tracked (always true)	Beverage product catalog with brand/flavor/size
lots	id, product_id, batch_number, production_date, expiration_date	Mandatory lot tracking for all beverages
stock	id, warehouse_id, location_id, product_id, lot_id, qty, reserved_qty	UNIQUE(w,l,p,lot) — core inventory
stock_movements	id, type ENUM, product_id, lot_id, from/to warehouse/location, qty,	APPEND-ONLY — immutable audit trail

	performed_by, created_at	
cost_layers	id, product_id, lot_id, qty, remaining_qty, unit_cost, received_at	FIFO cost tracking
users	id, email, password_hash, name, role ENUM, warehouse_id, is_active	4 roles: admin/warehouse_manager/picker/viewer
refresh_tokens	id, user_id, token_hash, expires_at	JWT refresh token storage

Key Design Decisions

Decision	Choice	Why
ORM vs Query Builder	sqlc + pgx	Compile-time type safety, full SQL power
Auth	JWT RS256	Stateless, mobile-friendly, no Redis
Audit Trail	Append-only stock_movements	Immutable source of truth
Multi-Warehouse	warehouse_id on rows	Flexible cross-warehouse queries
Costing	cost_layers with remaining_qty	FIFO — oldest layers consumed first
RBAC	Hard-coded in Go	4 roles, ~25 endpoints — no Casbin needed

9. Work Plan / Timeline

Phase	Week	Activities	Effort
Phase 0: Foundation	1	Go module + Fiber v3 server, PostgreSQL schema (all tables), sqlc codegen config, golang-migrate integration, env config, health check endpoint, Docker Compose	11.5h
Phase 1: Auth + Users	1–2	JWT auth (RS256), login/register/refresh, JWT middleware, role middleware (4 roles), user CRUD, password hashing, Vue login page, Pinia auth store, route guards	18h
Phase 2: Core Domain	2–4	Warehouse & location CRUD, product & category CRUD, barcode lookup, product search, inventory snapshot, low-stock alerts, inventory summary, Vue warehouse/product/inventory pages	35h
Phase 3: Stock Movements	4–5	Receive, pick, transfer, adjust — all transactional, FIFO cost layer consumption, movement history, Vue receive/pick/transfer/history forms	26h
Phase 4: Barcode Scanning	5–6	@teckel/vue-barcode-reader integration, camera scan component, scan-to-receive/pick flow, barcode label generation (boombuler), USB scanner composable	14h
Phase 5: Reporting	6–7	Inventory valuation (FIFO), movement summaries, product velocity, CSV export, enhanced dashboard (KPIs + charts), reports page, ApexCharts integration	15h

Phase 6: Polish	7–8	Input validation (Go + Zod), error standardization, rate limiting, logging middleware, Vue error boundaries, loading states, responsive design, Go/Vitest/Playwright tests	26h
----------------------------	-----	--	-----

Total Estimated Effort: ~145 hours (8–12 weeks single developer / 4–6 weeks 2-person team)

Milestone Summary

- Phase 0:** Server starts, DB connected, migrations run, /health returns 200
- Phase 1:** Users register/login, JWT tokens issued, protected routes enforce roles
- Phase 2:** Full warehouse/product/inventory CRUD working with Vue UI
- Phase 3:** All 4 movement types work, stock quantities correct, audit trail immutable
- Phase 4:** Scan barcode → product appears → receive/pick from same screen
- Phase 5:** Dashboard shows active stock, low-stock alerts, recent movements, valuation
- Phase 6:** Production-ready, tested, mobile-responsive

10. Budget and Resources

Development Costs (MVP — 3 months)

Category	Description	Estimated Cost
Development Tools	Go, Node.js, VS Code, Git, Docker	Free (open source)
Database Hosting	PostgreSQL on Railway / Supabase (dev tier)	\$0 – \$25/month
Backend Hosting	Go API on Fly.io / Railway (shared CPU)	\$0 – \$25/month
Frontend Hosting	Vue SPA on Vercel / Netlify (static hosting)	Free tier
Domain Name	Cloudflare / Namecheap	\$10 – \$15/year
SSL Certificate	Let's Encrypt / Cloudflare	Free
Barcode Library	boombuler/barcode (MIT License)	Free
Labor (Student Team)	Self-developed / academic project	Unpaid

Monthly Operating Costs (Post-Launch)

Resource	Estimated Monthly Cost
PostgreSQL (production tier, 2GB RAM)	\$25 – \$50/month
Backend hosting (2 vCPU, 4GB RAM)	\$25 – \$50/month
Frontend CDN + hosting	\$0 – \$20/month
Domain renewal	\$1/month
Monitoring (Sentry / Grafana free tier)	\$0

Total Monthly (estimated):	\$51 – \$121/month
-----------------------------------	---------------------------

Total MVP Cost

Development phase (3 months): \$0 – \$150 (hosting + domain only)

Post-launch monthly: \$51 – \$121/month

11. Risk Management

Risk	Prob	Impact	Mitigation Strategy
Beverage spoilage / expiry mismanagement — FEFO logic fails, causing expired stock to be picked instead of fresh stock	Medium	Critical	Rigorous FEFO unit tests with edge cases (same expiry, mixed lots); integration tests verifying pick order; expiry validation before pick commit
Stock quantity inconsistency — race condition between SELECT and UPDATE during concurrent movements	High	Critical	Use SELECT ... FOR UPDATE row-level locking within transactions; all movements in a single DB transaction with SERIALIZABLE isolation
FIFO cost layer misallocation — picks consuming wrong layers under concurrent load	Medium	High	Lock cost_layers rows with FOR UPDATE during pick; process layers in order; validate remaining_qty >= 0 after each consumption
Barcode scanner incompatibility — @teckel/vue-barcode-reader not working on certain devices	Medium	Medium	Fallback to manual SKU entry; USB scanner (keyboard wedge) as alternative input method; test on target devices early
Performance degradation — API slows down with 50k+ product rows and complex joins	Medium	High	Database indexing on all query fields; PrimeVue virtual scroll; paginate all list endpoints; query optimization with EXPLAIN ANALYZE
JWT token theft — attacker gains unauthorized access via	Low	Critical	Short-lived access tokens (15min); refresh token rotation;

stolen token			token revocation on role change; HTTPS enforcement
Data loss — PostgreSQL failure or accidental deletion	Low	Critical	Daily automated backups; WAL archiving; point-in-time recovery; append-only stock_movements means events are never deleted
Scope creep — features added beyond MVP causing delays	High	Medium	Strict MVP scope freeze after Phase 2; backlog all requests for Phase 2; weekly sprint reviews against original scope
Learning curve — team unfamiliar with Go + sqlc + Fiber stack	Medium	Medium	Allocate Phase 0 as learning buffer; create internal wiki/knowledge base; pair programming on complex SQL queries
Database migration failure — broken migration during deploy	Medium	High	Always test migrations on staging first; every migration has a verified down migration; Docker Compose for local testing
Mobile responsiveness issues — floor staff unable to use on tablets	Medium	Medium	Test on real mobile/tablet devices during Phase 6; use PrimeVue responsive utilities; separate mobile-optimized scan views

12. Expected Output / Deliverables

MVP Deliverables

1. Backend API Server (Go + Fiber v3)

- REST API with ~25 endpoints across 6 domains (auth, users, warehouses, products, inventory, movements, reports)
- JWT RS256 authentication with 4 role-based access levels
- PostgreSQL database with 10 tables, sqlc-generated type-safe queries
- Transactional stock movement processing (receive, pick, transfer, adjust)
- FIFO cost layer engine with automatic consumption
- Barcode label generation endpoint
- CSV report export
- Rate limiting and request logging middleware

2. Frontend SPA (Vue 3 + TypeScript + PrimeVue)

- Login/register pages with JWT auth flow
- Warehouse & location management with tree component
- Product catalog with PrimeVue DataTable (virtual scroll for 50k+ rows)
- Inventory dashboard with KPIs, charts, and low-stock alerts
- Stock movement forms — receive, pick (with barcode), transfer, history
- Camera-based barcode scanner component
- Reports page with valuation, velocity, and movement summaries
- User management (admin only)
- Responsive design for tablet/phone floor staff

3. Documentation

- API Documentation — all endpoints with request/response examples
- Database Schema Documentation — tables, relationships, constraints
- Deployment Guide — Docker Compose setup + cloud deployment instructions
- User Manual — guide for each role (admin, manager, picker, viewer)

4. Testing Suite

- Go integration tests — database layer, movement transactions
- Vue component tests (Vitest) — forms, tables, auth flow
- E2E tests (Playwright) — critical paths (login → scan → receive → verify stock)

5. Source Code & Infrastructure

- GitHub repository with full source code
- Docker Compose file for local development
- Database migration files (up + down)
- CI/CD pipeline configuration

13. Conclusion

The Warehouse Inventory Management System addresses a critical gap in how mid-size organizations manage inventory across multiple physical locations. By replacing error-prone spreadsheets and manual processes with a centralized, real-time system, it delivers immediate operational value: faster barcode-based operations, complete stock traceability, and accurate inventory valuation.

Key differentiators of this system include:

- **FEFO Picking:** The system automatically selects lots with the nearest expiration date during picks — critical in beverage distribution where product spoilage directly impacts profitability and regulatory compliance.
- **Expiry Management:** Automated alerts at 30/14/7 days before expiry give warehouse managers time to discount, redistribute, or dispose of aging stock — reducing write-offs and revenue loss.
- **Immutability:** The append-only `stock_movements` table serves as a permanent, verifiable audit trail — stock quantities can always be reconciled by replaying movements, essential for food & beverage regulatory audits.
- **Barcode-First Design:** Floor staff can scan a beverage product's GS1/EAN-13 barcode and complete a receive or FEFO pick operation in under 10 seconds, dramatically reducing operational friction in fast-moving distribution centers.
- **Temperature Zone Mapping:** Warehouses are organized by temperature zones (ambient, chilled, frozen), ensuring beverages are stored and tracked in appropriate conditions — reducing spoilage risk.
- **Clean Architecture:** The layered Go backend and modular Vue frontend make the system maintainable, testable, and extensible for future features like IoT temperature sensor integration or route optimization.

The system is built on a modern, production-proven stack (Go, PostgreSQL, Vue 3, TypeScript) that balances performance, developer productivity, and long-term maintainability. With an estimated 8–12 week timeline and minimal infrastructure costs

(\$0–\$150 during development), it represents a high-value, low-risk software project for any beverage distribution operation.

Upon completion, the Beverage Warehouse Inventory Management System will provide a solid foundation for beverage distributors to reduce spoilage, improve order accuracy, maintain regulatory compliance, and make data-driven inventory decisions.

14. Meet Our Team

[Add team member photos, names, roles, and quotes here following the template format]

	[Team Member 1] Project Manager "[Quote about the project]"
	[Team Member 2] Lead Developer "[Quote about the project]"
	[Team Member 3] Frontend Developer & UI/UX "[Quote about the project]"

15. Attachments / Annexes

- **Database Schema:** Full PostgreSQL DDL with all 10 tables, indexes, and constraints
- **API Specification:** OpenAPI/Swagger documentation for all ~25 endpoints
- **ER Diagram:** Visual entity-relationship diagram for the database
- **System Architecture Diagram:** 3-tier architecture with data flow
- **Barcode Scan Flow Diagram:** Camera/USB scan → lookup → movement → stock update
- **Auth Flow Diagram:** JWT login, refresh, and RBAC middleware pipeline